



四川大学
SICHUAN UNIVERSITY

面向长上下文建模的 量化索引残差动态记忆机制研究

硕士毕业设计开题答辩

吕昀键, 导师: 魏骁勇

四川大学计算机学院, 专业: 计算机科学与技术

2026.05.24

真实长文档任务: 从冗长文本中找回关键事实

Context

SUBSTANTIAL EQUIVALENCE DETERMINATION DECISION SUMMARY

A. 510(k) Number: K143329

B. Purpose for Submission: To obtain clearance for a new device

C. Measurand: A conserved multi-copy sequence of *Trichomonas vaginalis* genomic DNA

**D. Type of Test: Nucleic acid amplification assay
(Helicase-dependent Amplification, HDA)**

E. Applicant: Quidel Corporation

F. Proprietary and Established Names: Amplivue *Trichomonas* Assay

G. Regulatory Information: Classification: Class II; Panel: Microbiology

H. Intended Use: qualitative detection of *Trichomonas vaginalis* nucleic acids

真实文档中只有少数关键信息与问题相关, 其余内容都构成检索干扰。

长上下文场景中, 关键信息往往埋在很长、很密的文本中。

Query

What is the Type of Test?

Answer

Nucleic acid amplification assay
(Helicase-dependent Amplification, HDA)

模型不仅要读完整段上下文, 还要在后续查询时准确定位并读出对应事实。

长上下文模型真的“记住”了吗？

长文档问答、代码理解、长对话都依赖前文信息。长上下文能力的关键不只是“能输入多长”，更重要的是：后面能不能准确找回前面出现过的信息。

Input: 前文写入若干键值绑定, 后文只给出查询键

A 4 B 3 C 6 F 1 E 2 → A ? C ? F ? E ? B ?

Output: 模型必须按查询顺序读出对应值

4, 6, 1, 2, 3

形式上

记住前文的键值绑定

本质上

在大量干扰信息中准确读出

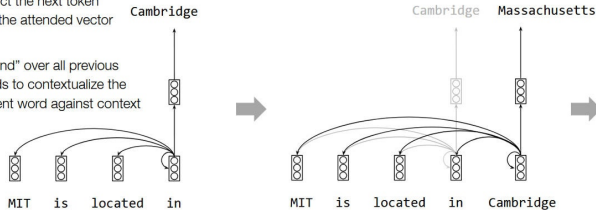
MQAR (Multi-Query Associative Recall): 检验长上下文中的多查询关联回忆能力。
关键不是只读完整个上下文, 而是在大量干扰信息中保持并读出正确的键值绑定。

全注意力: 记得准, 但代价高

标准 Transformer 可以让每个 token 直接访问所有历史 token。

Predict the next token
with the attended vector

"Attend" over all previous
words to contextualize the
current word against context



优势

直接比较所有历史位置
关联回忆能力强

代价

所有 token 两两比较
长序列下成本快速上升

Training ("Parallel Form")

$$\mathbf{O} = \text{softmax}((\mathbf{QK}^T) \odot \mathbf{M})\mathbf{V}$$

Inference ("Recurrent Form")

$$\mathbf{o}_t = \frac{\sum_{i=1}^t \exp(\mathbf{q}_t \mathbf{k}_i^T) \mathbf{v}_i}{\sum_{i=1}^t \exp(\mathbf{q}_t \mathbf{k}_i^T)}$$

Compute (Work)

$$O(L^2) \quad \text{☹️}$$

Memory

$$O(L) \quad \text{😊}$$

Steps

$$O(1) \quad \text{😊}$$

$$O(L^2) \quad \text{☹️}$$

$$O(L) \quad \text{☹️}$$

$$O(L)$$

高效模型：把历史压缩进有限状态

线性注意力, 状态空间模型, Mamba, GDN 等方法把历史信息压缩进可递推更新的状态.

状态递推与读出

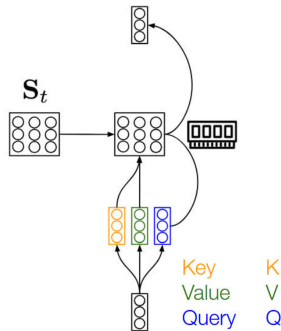
$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^T$$

$$\mathbf{o}_t = \mathbf{q}_t^T \mathbf{S}_t$$

共同思想

历史不再逐 token 保留
通过 $\mathbf{S}_t$ 递推写入
当前 query 从状态读出

	Fully Parallel Form	Chunkwise Parallel Form	Fully Recurrent Form
Compute	$O(L^2)$	$O(LC)$ ☺	$O(L)$
Memory	$O(L)$	$O(C)$	$O(1)$ ☺
Steps	$O(1)$	$O\left(\frac{L}{C}\right)$ ☺	$O(L)$



统一视角：高效模型都在维护可读写的状态

不同高效序列模型的递推形式不同, 但大多可以抽象为: **状态写入** + **状态读出**.

Model	Recurrence	Memory read-out
Linear Attention [48, 47]	$S_t = S_{t-1} + v_t k_t^\top$	$o_t = S_t q_t$
+ Kernel	$S_t = S_{t-1} + v_t \phi(k_t)^\top$	$o_t = S_t \phi(q_t)$
+ Normalization	$S_t = S_{t-1} + v_t \phi(k_t)^\top, z_t = z_{t-1} + \phi(k_t)$	$o_t = S_t \phi(q_t) / (z_t^\top \phi(q_t))$
DeltaNet [101]	$S_t = S_{t-1} (\mathbf{I} - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	$o_t = S_t q_t$
Gated RFA [81]	$S_t = g_t S_{t-1} + (1 - g_t) v_t k_t^\top, z_t = g_t z_{t-1} + (1 - g_t) k_t$	$o_t = S_t q_t / (z_t^\top q_t)$
S4 [32, 106]	$S_t = S_{t-1} \odot \exp(-(\alpha_t^\top) \odot \exp(A)) + B \odot (v_t \mathbf{1}^\top)$	$o_t = (S_t \odot C) \mathbf{1} + d \odot v_t$
ABC [82]	$S_t^k = S_{t-1}^k + k_t \phi_t^\top, S_t^v = S_{t-1}^v + v_t \phi_t^\top$	$o_t = S_t^v \text{softmax}(S_t^k q_t)$
DFW [63]	$S_t = S_{t-1} \odot (\beta_t \alpha_t^\top) + v_t k_t^\top$	$o_t = S_t q_t$
RetNet [108]	$S_t = \gamma S_{t-1} + v_t k_t^\top$	$o_t = S_t q_t$
Mamba [31]	$S_t = S_{t-1} \odot \exp(-(\alpha_t \mathbf{1}^\top) \odot \exp(A)) + (\alpha_t \odot v_t) k_t^\top$	$o_t = S_t q_t + d \odot v_t$
GLA [124]	$S_t = S_{t-1} \odot (\mathbf{1} \alpha_t^\top) + v_t k_t^\top = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	$o_t = S_t q_t$
RWKV-6 [79]	$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	$o_t = (S_{t-1} + (d \odot v_t) k_t^\top) q_t$
HGRN-2 [92]	$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t (\mathbf{1} - \alpha_t)^\top$	$o_t = S_t q_t$
mLSTM [9]	$S_t = f_t S_{t-1} + i_t v_t k_t^\top, z_t = f_t z_{t-1} + i_t k_t$	$o_t = S_t q_t / \max\{1, z_t^\top q_t \}$
Mamba-2 [19]	$S_t = \gamma_t S_{t-1} + v_t k_t^\top$	$o_t = S_t q_t$
GSA [131]	$S_t^k = S_{t-1}^k \text{Diag}(\alpha_t) + k_t \phi_t^\top, S_t^v = S_{t-1}^v \text{Diag}(\alpha_t) + v_t \phi_t^\top$	$o_t = S_t^v \text{softmax}(S_t^k q_t)$
Gated DeltaNet [125]	$S_t = S_{t-1} \left(\alpha_t (\mathbf{I} - \beta_t k_t k_t^\top) \right) + \beta_t v_t k_t^\top$	$o_t = S_t q_t$

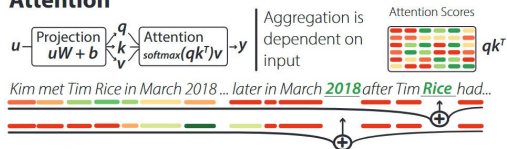
差异在于如何更新状态; 共同点在于都把长历史压缩到可递推的记忆状态中.

压缩的代价：容量不足会损伤检索

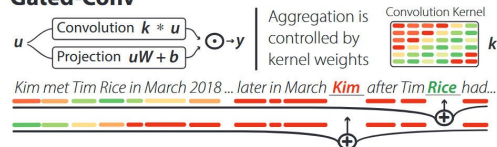
有限状态压缩把多组键值绑定写入同一个状态, 容量不足时查询读出会发生混叠.

机制直觉: 聚合会混入无关信息

Attention



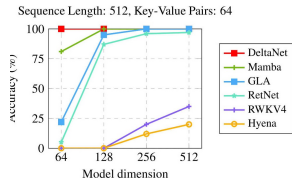
Gated-Conv



性能证据: Retrieval 受损

Model	PPL ↓	LM Eval ↑	Retrieval ↑
Transformer++	16.9	50.9	41.8
RetNet	18.6	48.9	30.6
Mamba	17.1	50.0	27.6
Gated Linear Attention	17.2	51.1	37.7
DeltaNet	16.9	51.6	34.7

容量证据: 维度越大, 记忆越强



高效模型的问题不是不能压缩, 而是压缩后还能不能准确保存并读出键值绑定.

现有高效模型的研究缺口

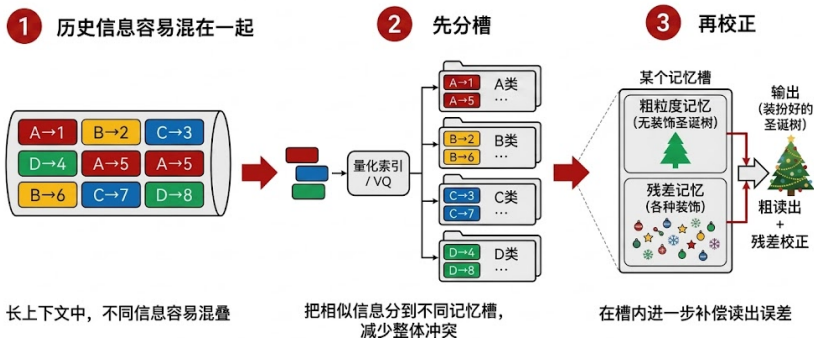
已有高效长上下文模型分别从遗忘, 改写和索引三条路线缓解状态压缩问题.

方法思路	代表模型	主要解决什么	仍然不足
动态遗忘	GLA / Mamba	控制历史信息保留, 避免无关状态长期累积	不直接解决槽内混叠
定向改写	DeltaNet / GDN	用输入相关的更新规则 改写旧记忆	缺少显式分槽组织
量化索引	Transformer-VQ / LongVQ	VQ 压缩全局历史, 线性化长序列注意力	槽内混合误差 校正不足
本课题关注	Flash-VQRM	有限状态混叠 与槽内读出误差	待系统验证

关键缺口: 分槽之后, 同一记忆槽内的读出误差仍需要进一步校正.

我们的思路: 先分槽, 再校正

本课题将该结构命名为 **Flash-VQRM** (Flash Vector-Quantized Residual Memory Attention): 量化索引
先把历史分到记忆槽, 槽内残差再校正粗粒度读出误差。

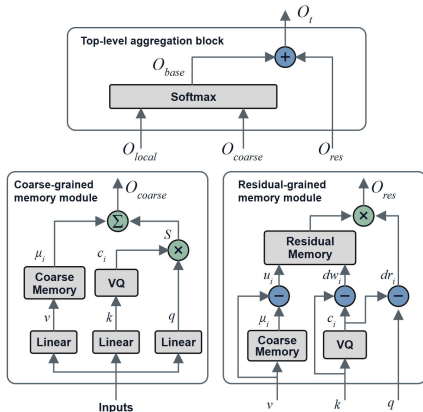


核心目标: 减少有限状态中的混叠, 并补偿分槽后的读出误差

设计直觉: 分槽减少跨类冲突, 残差补偿槽内误差。

方法概览: Flash-VQRM

模型把一次状态读写拆成三部分: 分槽统计, 残差校正, 和 顶层融合.



Coarse memory: 先分槽统计

VQ 将相近键路由到同一记忆槽,
coarse memory 保存槽级粗粒度历史.

Residual memory: 再校正误差

residual memory 记录槽内细节偏差,
用于补偿粗粒度读出的误差.

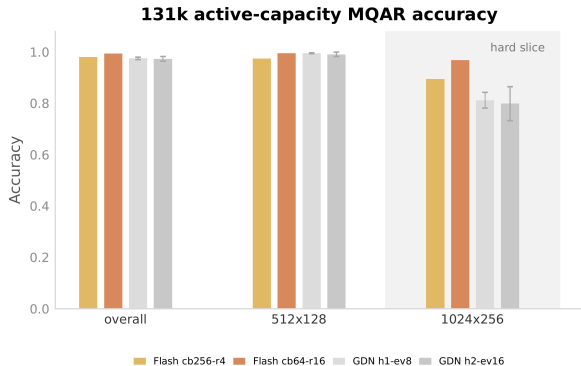
Aggregation: 融合三路输出

顶层聚合把 O_{local} , O_{coarse} , O_{res} 合
成为最终输出 O_t .

关键流程: VQ 分槽组织历史, residual memory 负责槽内误差补偿.

MQAR 预研: 131k 容量下高负载更稳

同为 **131k active capacity**, Flash-VQRM 在 hard slice 1024x256 上保留更高准确率。



131k active-capacity MQAR summary

config	cap	params	overall	128	256	512	1024
Flash cb256-r4	131.1k	1.183M	0.980	0.995	0.992	0.975	0.895
Flash cb64-r16	131.1k	1.160M	0.994	0.996	0.998	0.995	0.969
GDN h1-ev8	131.1k	1.368M	0.975	1.000	1.000	0.995	0.812
GDN h2-ev16	131.1k	1.635M	0.974	1.000	1.000	0.991	0.799

主要读数 (1024x256)

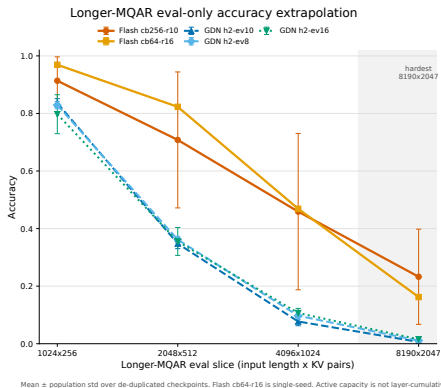
Flash cb64-r16: **0.969**.

GDN h1-ev8: 0.812, GDN h2-ev16: 0.799.

结论: 同等容量下, **Flash-VQRM** 在高负载 **hard slice** 上更稳, 说明分槽加校正能缓解状态压缩误差。

长度外推预研: Flash-VQRM 衰减更慢

Longer-MQAR 不重新训练, 直接评估超过训练长度后的关联回忆能力.



Longer-MQAR eval accuracy

config	cap	params	1024	2048	4096	8190
Flash cb256-r10	327.7k	1.184M	0.914	0.708	0.459	0.233
Flash cb64-r16	131.1k	1.160M	0.969	0.823	0.469	0.162
GDN h2-ev16	131.1k	1.635M	0.797	0.355	0.107	0.015
GDN h2-ev10	81.9k	1.435M	0.836	0.348	0.077	0.006
GDN h2-ev8	65.5k	1.368M	0.829	0.362	0.097	0.011

主要读数

2048x512: Flash cb64-r16 为 **0.823**, GDN 为 0.355;
4096x1024: Flash cb64-r16 为 **0.469**, GDN 为 0.107.

结论: 不重训直接外推时, Flash-VQRM 在 2048 到 8190 slice 上衰减更慢, 保留更强关联回忆能力.

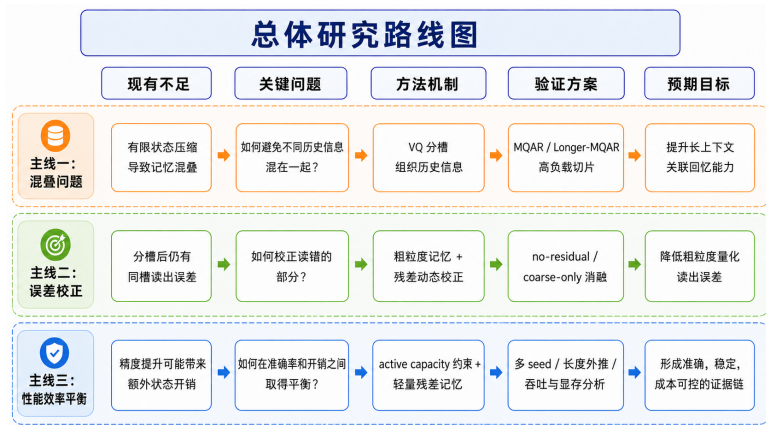
本课题拟解决的三个关键问题

前面预研说明方法方向可行, 后续重点是回答三个机制性问题.

关键问题	对应机制	验证方式
Q1: 如何减少记忆混叠?	量化索引分槽, 让不同历史信息分开组织.	MQAR 高负载切片, capacity-fair 强基线对比.
Q2: 同槽读出误差如何校正?	residual memory 记录槽内偏差, 补偿粗粒度读出误差.	coarse-only / no-residual 消融, cb / rank 变化分析.
Q3: 如何兼顾性能与效率?	限制 active capacity 与状态开销, 提高单位容量收益.	准确率, 吞吐和显存统一评估, 结合 Longer-MQAR 分析.

核心闭环: 先解决有限状态混叠, 再校正槽内误差, 最后评估性能与效率的可用平衡.

总体研究路线图



路线重点：围绕混叠抑制，槽内误差校正和性能效率平衡，形成 **Flash-VQRM** 的机制验证闭环。

感谢各位老师



四川大學
SICHUAN UNIVERSITY

请各位老师批评指正